Imitation Game: Reproducing Deep Learning Bugs Leveraging an Intelligent Agent

Mehil B Shah
Dalhousie University
Halifax, Canada
shahmehil@dal.ca

Mohammad Masudur Rahman
Dalhousie University
Halifax, Canada
masud.rahman@dal.ca

Foutse Khomh
Polytechnique Montreal
Montreal, Canada
foutse.khomh@polymtl.ca

Abstract

Despite their wide adoption in various domains (e.g., healthcare, finance, software engineering), Deep Learning (DL)-based applications suffer from many bugs, failures, and vulnerabilities. Reproducing these bugs is essential for their resolution, but it is extremely challenging due to the inherent nondeterminism of DL models and their tight coupling with hardware and software environments. According to recent studies, only about 3% of DL bugs can be reliably reproduced using manual approaches. To address these challenges, we present RepGen, a novel, automated, and intelligent approach for reproducing deep learning bugs. RepGen constructs a learning-enhanced context from a project, develops a comprehensive plan for bug reproduction, employs an iterative generate-validate-refine mechanism, and thus generates such code using an LLM that reproduces the bug at hand. We evaluate RepGen on 106 real-world deep learning bugs and achieve a reproduction rate of 80.19%, a 19.81% improvement over the state-of-the-art measure. A developer study involving 27 participants shows that RepGen improves the success rate of DL bug reproduction by 23.35%, reduces the time to reproduce by 56.8%, and lowers participants' cognitive load.

CCS Concepts

• **Software and its engineering → Software testing and debugging**.

Keywords

Deep learning bugs, deep learning bug reproduction, automated debugging, LLM-powered agents, code generation, machine learning systems, software testing and debugging

ACM Reference Format:
Mehil B Shah, Mohammad Masudur Rahman, and Foutse Khomh. 2026. Imitation Game: Reproducing Deep Learning Bugs Leveraging an Intelligent Agent. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3787795

1 Introduction

Artificial Intelligence (AI) has been widely adopted in many application domains, including software engineering [45, 46], autonomous vehicles [27], healthcare [64], finance [10], and cybersecurity [16]. The global market share of AI software reached $34.8 billion in 2023 and is projected to grow up to $360 billion by 2030 [26]. Over 67% of top-performing companies have incorporated AI in their business solutions, and 97% of Fortune 500 companies have invested in AI technologies [58], indicating their significance. However, software applications empowered by Deep Learning (DL), the underlying technology behind current AI systems, remain prone to bugs, faults, and vulnerabilities, which could lead to major consequences (e.g., system crashes) and catastrophic failures (e.g., autonomous vehicle accidents) [69]. Unlike the bugs in traditional, developer-written software, the bugs in DL software are inherently challenging due to several factors. First, they are often non-deterministic due to randomness in model training, i.e., random weight initialization of the model layers [52]. Second, DL models perform high-dimensional tensor operations and suffer from a lack of interpretability, making their encountered bugs opaque [46]. Finally, these bugs also have multi-faceted dependencies on hardware (e.g., GPU), underlying frameworks (e.g., PyTorch, TensorFlow) [81], and data pipelines, making them highly complex [62].

To resolve DL bugs, software developers must first systematically reproduce them on their local machines. Without a reproduction, they cannot confirm the presence of a bug or diagnose its root cause. However, reproduction of DL bugs can be effort-intensive, time-consuming, and frustrating due to various technical challenges. They include intricate data pipelines, hardware dependencies, and variations in software frameworks and library versions. Even when a bug is reproducible, developers frequently need to engage in trial-and-error, carefully tune environmental settings, and reason about the contextual factors that may influence the behaviour of DL programs, all of which can be tedious and error-prone [54]. Developers also face the challenge of missing or incomplete information when attempting to reproduce bugs from issue reports [71]. Reports may lack crucial details of a bug and omit relevant data or code snippets. In such cases, even experienced developers must spend substantial time reconstructing the missing detail of a bug, the target environment, and iteratively testing hypotheses to reproduce the erroneous behaviour [32, 82]. These difficulties make the manual process of reproducing DL bugs highly inefficient and error-prone. Thus, there is a strong need for techniques that can *accelerate and systematize* the reproduction of DL bugs, reducing developer effort while increasing the reliability of DL systems.

Over the last two decades, there have been many techniques to support developers in localizing [20, 47, 60], reproducing [40, 53], and correcting software bugs [17, 44, 78]. However, they are not sufficient since they were not designed to tackle deep learning specific challenges. They might fail to effectively reproduce DL bugs or may do so inefficiently due to the following limitations:

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
ICSE '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/2026/04
https://doi.org/10.1145/3744916.3787795

(a) Limited contextual understanding: Existing techniques (e.g., RecDroid [84], ReBL [73], and AdbGPT [24]) analyze bug reports and GUI applications and leverage UI interactions or code execution paths to reproduce software bugs. In contrast, DL bugs go beyond source code and can originate from multiple artifacts scattered across different parts of the ML pipeline, including model architectures, training datasets, preprocessing scripts, hyperparameters, underlying frameworks, and hardware (e.g., GPU) [63]. Collecting and correlating information from these scattered artifacts systematically is challenging, as each may contribute in subtle ways to the manifestation of the bug. Existing techniques lack mechanisms to identify or utilize these DL-specific contextual clues, even when they are present in bug reports or code, which makes these approaches inefficient or insufficient for reliably reproducing DL bugs.

(b) Heavy reliance on GUI-specific elements: Many existing techniques—RecDroid [84], ReBL [73], AdbGPT [24], ReActDroid [33], LLMDroid [70]—capture well-defined states of GUI-based applications using record-and-replay methods [31] and generate deterministic event sequences to reproduce software faults. However, DL bugs originate from various stochastic processes such as model training, hyperparameter optimization, and data pipeline execution, which do not involve any GUI interactions [63]. Thus, existing techniques relying on GUI interactions (e.g., mouse clicks, or screen capture) are fundamentally misaligned with DL-based systems, which makes them less effective for reproducing DL bugs.

(c) Challenges with "silent" bugs: Traditional techniques often verify the reproduction of a software bug by checking for crashes or undesirable program states [53, 66]. However, DL bugs can manifest themselves in non-conventional ways, such as increased losses during model training or extremely poor performance during model inference. Bugs with such symptoms are called *silent* bugs in literature [67]. According to a recent study [67], 72.8% of developers reported difficulty in resolving silent bugs, which highlights the persistent challenges of these bugs. Thus, existing techniques [53, 66] relying on crashes or state-specific symptoms might be inadequate for accurately reproducing silent bugs from DL systems.

Given these limitations of existing techniques, we present *RepGen*, an automated, intelligent approach for reproducing DL bugs, with a specific focus on accelerating and systematizing their reproduction. Unlike the existing techniques, our approach leverages a *learning-enhanced context* for code generation and approximates *fault symptoms of the code using LLMs*, thereby improving the speed, consistency, and reliability of DL bug reproduction. First, our technique builds a learning-enhanced context against a DL bug by capturing relevant code containing model training loops and their dependencies from a project. Second, using the context, RepGen generates a comprehensive plan for bug reproduction, which captures the environment setup, model training mechanism, and inference pipeline. Finally, both the context and the plan are passed to the LLM-powered agent, which runs an iterative *generate–validate–refine* cycle to generate code that can reproduce the bug at hand. Our agent not only evaluates the code using standard mechanisms (e.g., compiler feedback, static analysis, relevance feedback) [61, 68, 75] but also refines the code and validates its fault symptoms using an LLM (e.g., Qwen2.5-Coder-7B) to enhance its effectiveness in reproducing DL bugs. To the best of our knowledge,

```
Title: attn_num_mem_kv > 0 and attn_one_kv_head = True error
Description: If you set both attn_num_mem_kv & attn_one_kv_head, you get an error

Repro:
Im = TransformerWrapper (num_tokens = 32, max_seq_len = 0,  num_memory_tokens = 20,
                         attn_layers = Decoder (dim = 512, depth = 4, heads = 4,
                         rotary-pos_emb = True, attn_flash = True,
                         attn_onnable = True, attn_num_mem_kv = 20,
                         attn_one_kv_head = True))
x = torch.randint(0, 32, (8, 1024))
logits = lm(x)
print(x.shape, x.dtype)
print(logits.shape, logits.dtype)
```

Figure 1: Bug #228 in X-Transformer; red boxes indicate incorrect or missing parameters in the user-provided snippet.

RepGen is the first technique to offer a unified, intelligent solution to systematically reproduce DL bugs, making our work *novel*.

We evaluated RepGen using a comprehensive dataset of 106 real-world deep learning bugs collected from 16 GitHub projects. Our technique successfully reproduced 85 (80.19%) bugs and outperformed eight LLM-only baselines (e.g., GPT 4.1, Llama3, DeepSeek-R1, Qwen3, Qwen2.5 and their variants), indicating its superiority. A controlled developer study with 27 participants further validated our results, showing that participants can reproduce ≈23.35% more bugs when assisted by RepGen and can save ≈56.8% time during bug reproduction. All these findings suggest a strong potential of our technique for supporting the systematic and efficient reproduction of deep learning bugs.

We thus make the following contributions in this paper:

(a) A novel, intelligent technique - RepGen - to support the reproduction of deep learning bugs, emphasizing the acceleration and systematization of bug reproduction.

(b) A novel agentic workflow that builds a comprehensive, learning-enhanced context, develops a targeted reproduction plan, and leverages a feedback-driven reproduction agent that can generate high-quality code capable of reproducing DL bugs efficiently.

(c) A comprehensive experiment comprising an empirical evaluation of RepGen, an ablation study targeting 13 major components, comparisons with eight baselines, and a developer study involving 27 participants, demonstrating the benefits of our technique.

(d) A carefully curated benchmark dataset containing 106 bug reports and associated artifacts (e.g., code examples, configuration files) from 16 real-world deep learning projects.

(e) A replication package [9] containing our implementation, dataset, and experimental results to enable future research.

2 Motivating Example

To demonstrate the effectiveness of our technique, let us consider the bug in Fig. 1 (Bug #228, X-transformer). The bug occurs in the `TransformerWrapper` class when two specific parameters – `attn_num_mem_kv` and `attn_one_kv_head` – are initialized simultaneously. In particular, when the model uses a single shared key-value head (`attn_one_kv_head = True`) and simultaneously allocates the memory for additional key-value pairs (`attn_num_mem_kv = 20`), it leads to an incompatible configuration for the attention model, which causes a `ValueError`.

As shown in Fig. 1, the bug report provides a partially complete code snippet and lacks necessary details (e.g., required libraries). Reproducing such bugs manually is possible, but it requires developers to identify missing dependencies, understand parameter interdependencies, configure the target environment, and iteratively test for the reported faulty behaviour, which can be time-consuming

```python
import torch
from x_transformers import TransformerWrapper, Decoder
import traceback

print("\nTesting with both attn_num_mem_kv=20 and attn_one_kv_head=True:")
try:
    model = TransformerWrapper(num_tokens=32, max_seq_len=1024, num_memory_tokens=20,
            attn_layers=Decoder(dim=512, depth=4, heads=4, rotary_pos_emb=True,
            attn_flash=True, attn_onnxable=True, attn_num_mem_kv=20,
            attn_one_kv_head=True))
    x = torch.randint(0, 32, (8, 128))
    logits = model(x)
    print("✗ Unexpected success: Model with conflicting parameters works")
except Exception as e:
    print(f"✓ Expected error: {type(e).__name__}")
    traceback.print_exc()
    print("\n✓ Bug successfully reproduced: Conflict between parameters")
```

Figure 2: Reproduction code generated by RepGen, with green boxes highlighting additions that make the script verifiable and executable.

and error-prone. Existing techniques for bug reproduction - RecDroid [84], AdbGPT [24] - rely heavily on GUI-based interactions and event sequences, making them non-applicable to DL models operating through training data, model structure, hardware configurations, and underlying frameworks. Hence, they naturally fail to reproduce such bugs. On the other hand, ChatGPT, a high-performing LLM, accepts the bug report (Fig. 1) and generates a non-compilable code snippet[1] with incorrect import statements.

In contrast, our technique, RepGen, systematizes the bug reproduction process and provides a code example (Fig. 2) that successfully reproduces the bug. First, RepGen constructs a *learning-enhanced context* by combining the code retrieved from the `TransformerWrapper` and `Decoder` classes with the useful information from the bug report (e.g., erroneous behaviors). Second, RepGen leverages the context and formulates an appropriate plan to reproduce the bug. Finally, the context and the plan are passed to our reproduction agent. The reproduction agent then generates an initial version of code that misses required import statements, as detected by PyLint. Using PyLint's feedback, our reproduction agent generates a more complete and relevant code snippet, but it still lacks an important configuration needed to reproduce the bug (e.g., both `attn_num_mem_kv` and `attn_one_kv_head` should be initialized simultaneously). This issue was detected by our reproduction agent, suggesting that the code missed the symptoms mentioned in the bug report. By incorporating this feedback into the regeneration process, RepGen then generates a complete code example that can reproduce the bug (Fig. 2).

In short, by leveraging a learning-enhanced context, comprehensive planning, multi-phase feedback, and iterative refinement, our technique enables the automated and structured reproduction of DL bugs that would otherwise require substantial manual effort. RepGen completes this process in ≈5 minutes, demonstrating its ability to accelerate and systematize bug reproduction. This also highlights our technique's potential for improving developer productivity and ensuring consistent, verifiable reproduction of DL bugs.

3 RepGen

Fig. 3 shows the schematic diagram of our proposed technique – RepGen– for DL bug reproduction. We discuss the major steps of our methodology as follows.

3.1 Construction of Learning-Enhanced Context

Since we use an LLM to generate code capable of reproducing bugs from real-life projects, we need project-specific context. We thus leverage a hybrid retrieval framework and construct a learning-enhanced context against a bug report as follows (Step 1a, Fig. 3).

3.1.1 Pre-Retrieval. Before performing any retrieval, we apply a series of preprocessing steps to the codebase as follows.

(a) Semantic chunking: We break each source code file into smaller, semantically meaningful segments through code chunking [43]. It is essential since DL codebases often contain long, complex files spanning hundreds of lines. We employ Abstract Syntax Tree (AST) parsing, identify natural partitioning points (e.g., method boundary, class declaration), and break each file into smaller chunks that are structurally correct and semantically relevant.

(b) Indexing: After splitting the code into chunks, we index them for efficient search and real-time analysis. We construct two complementary indices for efficient retrieval of relevant code snippets: a sparse retrieval index that can be leveraged by the BM25 ranking function [59] and a dense retrieval index that can be leveraged for embedding-based semantic retrieval.

3.1.2 Code Snippet Retrieval. To retrieve relevant code against a bug report, we design a hybrid retrieval approach leveraging the above indices as follows.

(a) BM25 search: Best Matching 25 (BM25) is a probabilistic algorithm for similarity matching that incorporates document length normalization and term frequency saturation. It can identify code chunks that contain exact matches of API calls, error messages, or specific identifiers found in a bug report. Given a query Q derived from the bug description, we compute the BM25 score for a code chunk D [59]. We also normalize the BM25 scores across all the chunks for an effective analysis [79].

(b) ANN search: Approximate Nearest Neighbour (ANN) search leverages semantic proximity between two items within the semantic space. We use an ANN search to capture the conceptual or semantic relationship between code chunks and bug reports that may not be evident from their contents. Using a code-language model[2], we encode each bug report and code chunk into dense vectors and normalize them. Then, to measure the similarity between the bug report ($\mathbf{q}$) and code ($\mathbf{d}$), we compute their angular similarity [1].

(c) Hybrid scoring: To leverage the complementary strengths of both approaches, we combine the normalized scores from the BM25 Search and ANN Search using the formula below:

$$\text{score}_{\text{h}}(D) = (1-\alpha) \cdot \text{BM25}_{\text{norm}}(Q, D) + \alpha \cdot \text{sim}_{\text{angular}}(\mathbf{q}, \mathbf{d})$$

After iterative hyperparameter tuning, we set α to 0.55. We then rank the code snippets by their hybrid scores and select the top K (e.g., 20) for further re-ranking.

(d) Reranking: After retrieving a short list of code snippets above, we employ a cross-encoder[3] to re-rank them. The *ms-marco-MiniLM-L12-v2* cross-encoder demonstrates a strong capability in understanding texts and code, making it a suitable choice for our approach. First, MiniLMv2, distilled from the BERT and RoBERTa architectures, inherits their strong performance on code-specific tasks [22, 41, 72]. Second, its fine-tuning on the MS Marco dataset [14], derived from Bing search queries, provides a comprehensive understanding of human-generated natural language. Unlike the above retrieval methods, which rely on lexical or semantic matching, the

[1] http://bit.ly/462AOlj

[2] http://bit.ly/3IwN0kk

[3] http://bit.ly/454NWFo

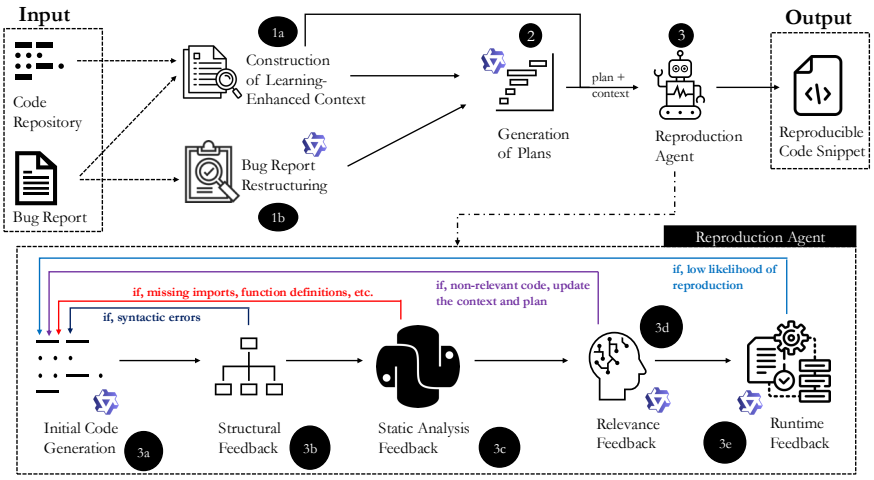

Figure 3: Schematic diagram of our proposed technique - RepGen

cross-encoder jointly encodes both the bug report and code snippet, allowing its attention mechanisms to detect their exact relationships. It can also identify subtle but important connections that the simpler retrieval methods might miss.

Given a bug report Q and code snippet D, the cross-encoder computes a relevance score, as follows:

$$\text{score}_{\text{cross}}(Q, D) = \text{CrossEncoder}([\text{CLS}] \oplus Q \oplus [\text{SEP}] \oplus D \oplus [\text{SEP}])$$

where $\oplus$ denotes concatenation with special tokens (e.g., CLS, SEP), and the model analyzes cross-attention between all tokens to produce a score between 0 and 1. Based on the cross-encoder scores, we re-rank the top K snippets (e.g., 20) for further processing.

(e) Capturing dependencies: Deep learning pipeline involves multiple components, and their bugs could be multifaceted. For example, a bug in a model training loop might depend on the custom layer's definition, data preprocessing functions, and custom loss functions defined in separate modules. Therefore, for each retrieved snippet D_i, we parse its AST and identify the corresponding dependencies (e.g., imported modules, referenced variables). The dependency resolution ensures that all necessary code components are captured for subsequent analysis.

3.1.3 Context Construction. Once relevant code snippets and their dependencies are captured, we construct a unified context that can help LLMs generate appropriate code reproducing deep learning bugs. To achieve this, we perform several analyses using the retrieved code and bug report as follows.

(a) Module-centric partitioning & retrieval: We organize the retrieved code snippets based on their source modules. Our idea was to select the modules containing the most relevant code to the bug. We thus map the code snippets to their corresponding system paths and organize them in modules:

$$M_k = \text{ResolveModule}(D_i), \quad k \in \{1, ..., N\}, N \leq 5$$

where D_i refers to an individual code snippet, and `ResolveModule` is the function that assigns it to a specific module, M_k. In this step, we cluster the code snippets according to their respective modules and determine each module's priority based on its members' maximum score from the re-ranking step. In other words, a module with a single highly relevant snippet will be prioritized over one with numerous less relevant snippets. After determining the module priority, we select the top five modules for subsequent analysis.

(b) AST-driven training loop extraction: Through a comprehensive analysis of TensorFlow and PyTorch documentation, we identified eight framework-specific heuristics for detecting model training loops. They include method calls like `model.fit()`, `optimizer.step()`, and custom training loop patterns. These heuristics are available in our replication package [9]. Using these heuristics and Abstract Syntax Tree (AST)-based parsing, we capture the potential training loops from each module:

$$\mathcal{L}_k = \bigcup_{D_i \in M_k} \text{ExtractLoops}(\text{AST}(D_i))$$

This extraction systematically identifies various components of a training loop, including forward/backward passes, gradient computations, optimizer steps, loss calculations, and data loader interactions. These specific components are crucial because they accurately distinguish actual training loops from other code snippets (e.g., model definitions, preprocessing scripts). The identified training loops are then used for subsequent steps.

(c) Training loop ranking: Given that a module may contain multiple training loops, we need to identify those most relevant to a reported bug. We employ the same cross-encoder[4] as the re-ranking step to assess the relevance of each training loop to the bug report, leveraging its strong understanding of both code and natural language text. Given a training loop L_j, and the bug report query (Q), the cross-encoder computes the relevance score as follows:

$$\text{score}_{\text{loop}}(L_j) = \text{CrossEncoder}([\text{CLS}] \oplus Q \oplus [\text{SEP}] \oplus L_j)$$

where $\oplus$ denotes concatenation with special tokens (e.g., CLS, SEP). From each module, only one loop, specifically the one with the highest relevance score, is selected for the subsequent steps.

(d) Construction of learning-enhanced code context: Once training loops are collected, we construct the code context for every module by combining its training loop, relevant code snippets, and dependencies as follows:

[4]http://bit.ly/454NWFo

$$C_k = \underbrace{L_k}_{\text{training loop}} \cup \underbrace{\bigcup_{D_i \in \text{Top}_5(M_k)} D_i}_{\text{relevant code snippets}} \cup \underbrace{\text{DepGraph}(D_i)}_{\text{dependencies}}$$

$$C_{\text{out}} = [C_1, ..., C_K]$$

Here, C_k represents each of the top K (e.g., 5) contexts constructed. Our reproduction agent leverages each of these contexts until it succeeds in generating the code capable of reproducing a DL bug. By combining the most relevant training loops, code snippets, and their dependencies, we thus construct a learning-enhanced code context that offers useful information to our LLM-based reproduction agent (Step 1a, Fig. 3).

3.2 Bug Report Restructuring

Using an LLM, we restructure each bug report and extract its useful information (e.g., erroneous behaviours) (Step 1b, Fig. 3), as follows.

First, we analyze the report title and opening paragraphs to extract the core problem statement about a bug. Second, we capture the technical details, including stack traces, error messages, performance metrics, and resource utilization patterns, to form the *Observed Behaviour* section. Third, we also capture the intended system behaviour from report text, API specifications, and baseline metrics, and construct a detailed *Expected Behaviour* section. Finally, RepGen synthesizes all gathered information into a detailed sequence of steps that specifies environment configurations, hardware requirements, dataset specifications, and training parameters, and constructs the steps for reproducing the bug. If any steps are explicitly mentioned in a bug report, they are leveraged by the LLM. Otherwise, the LLM reasons through the gathered information and generates a set of candidate steps. Then all three items are used by the LLM to restructure and enhance the bug report.

3.3 Plan Generation

To reproduce any deep learning bug, we need a comprehensive plan. To construct the prompt for generating plans, we utilize the existing guidelines for self-planning [38], which have demonstrated good performance in code generation. Self-planning has been shown in existing literature [38] to generate plans that produce more correct, reliable, and robust code. Using the prompt and an LLM, RepGen generates a plan that breaks the task of bug reproduction (Step 2, Fig. 3) into small, manageable steps. The plan guides our agent through code generation, ensuring all necessary components, configurations, and validation steps are considered. We adopt the following steps to generate our plan:

First, we identify the key components (e.g., training loops, module dependencies) and their interactions within each context using the LLM. Second, we capture the reproduction steps and numerical parameters from the bug report. Third, we structure the plan into modular, verifiable stages to enable systematic validation and control during bug reproduction attempts. In particular, it incorporates output assertions, resource monitoring, and error verification steps into the plan. Finally, we capture the plan as a structured sequence of steps to ensure seamless integration within the workflow. As a part of this step, we generate five plans (one for each context) and then pass the enriched context and its associated plan to the reproduction agent. Fig. 4 shows an example plan generated by our approach.

```
"Import necessary modules from the provided code snippet, including
dataclasses, functions, and PyTorch components.",
"Define the configuration parameters for model in dictionary format similar to
Configs class instantiation in the main function.",
"Create an instance of the Trainer class by passing the configuration
dictionary as an argument.",
"Call the train method on the trainer instance to start the training process.",
"Monitor the tracking metrics during training, such as loss and global step
count, using functions like tracker.add_scalar.",
"Assert that the loss value is being updated and tracked during each epoch of
training.",
"After completing one full cycle through the epochs (i.e., after calling
trainer.train())",
"Save the model checkpoint using experiment.save_checkpoint()",
"Validate that the saved checkpoint file exists and has a non-zero size,
indicating successful saving of the model weights."
```

Figure 4: Plan generated by RepGen

3.4 Code Generation using LLM

To execute the above plan and generate code capable of reproducing DL bugs, we introduce a specialized LLM-powered agent (Step 3, Fig. 3). To avoid hallucinations, our agent combines LLMs with an enriched context and a robust, multi-stage validation framework. Such a framework enables an iterative refinement process, guided by comprehensive feedback, ensuring that the generated code accurately and reliably reproduces a bug. We discuss the workflow of our bug reproduction agent below.

Candidate Code Generation: Our agent generates code by considering the restructured bug report, available code context, and our generated plan (Step 3a, Fig. 3). This initial code is likely to contain the critical components (e.g., required import statements, model architecture, training pipeline). It serves as a foundation that can be evolved to generate the final code snippet reproducing a bug.

Capturing feedback on generated code: To ensure syntactic correctness of our generated code, we employ AST-based *structural validation* (Step 3b, Fig. 3). It serves as the first line of defence against syntactic errors, ensuring adherence to language grammar rules. Code that fails syntactic checks undergoes regeneration, ensuring that only syntactically sound code proceeds to subsequent stages. We employ PyLint-based *static analysis* (Step 3c, Fig. 3) to identify potential vulnerabilities or quality issues in the syntactically correct code. It also looks for common programming errors, including missing variable definitions, incorrect import statements, invalid function calls, and type inconsistencies. The detailed feedback from this step triggers further refinement of the code by the LLM, ensuring adherence to Python best practices and minimizing potential runtime failures. Beyond syntactic correctness and completeness, we further incorporate mechanisms to ensure the relevance and effectiveness of the generated code. To avoid hallucinations and ensure relevance to the original bug report, we capture *intelligent relevance feedback* against the generated code by leveraging an existing method [61] (Step 3d, Fig. 3). The mechanism evaluates the semantic alignment between the generated code and the bug report. If the intelligent relevance feedback indicates the irrelevance of the code, we switch to the next available context. This step is crucial given the tendency of LLMs to generate plausible but irrelevant code. Besides static feedback mechanisms above, we introduce a novel *runtime feedback* mechanism that evaluates the likelihood of our generated code successfully reproducing a reported bug (Step 3e, Fig. 3). We capture such feedback through a systematic, multi-step process as follows. First, we approximate the final output and state of the generated code by leveraging the code understanding and reasoning capability of LLMs [21]. Second, we leverage the LLM to map the program state (e.g., high loss values) to known categories of DL bugs from an established taxonomy (e.g., incorrect

loss function), which we injected into the context [34]. If a direct match is not found in the existing taxonomy, we approximate the category of the bug based on its pre-trained knowledge. Third, the LLM leverages the taxonomy of symptoms [19, 67] provided in the prompt to derive the symptoms of the potential bugs in the code. If the symptoms are not explicitly covered in the existing taxonomy, LLM approximates the symptoms based on its pre-trained knowledge. Finally, the symptoms of the generated code are compared to those in the bug report using an LLM, which provides a similarity score. A high similarity score indicates a strong likelihood of successful bug reproduction, whereas a low score triggers an iterative refinement process. In the case of a low score, the LLM regenerates the code by incorporating the feedback on the symptoms of the code and the bug report. This dynamic feedback loop ensures continuous improvement of the generated code and its chance of reproducing the bug. Furthermore, if the agent fails to generate bug-reproducing code within five attempts, the system switches to the next available code context. Our choice of five attempts is inspired by relevant literature [74]. In short, our approach systematizes DL bug reproduction by delivering a comprehensive code snippet that quickly and reliably reproduces the bug.

3.5 Implementation Details

We implement RepGen using carefully selected open-source LLMs that meet three important requirements: demonstrated excellence in code-related tasks, support for efficient quantization, and robust community maintenance. We leverage *Qwen2.5-7B* for text processing tasks such as bug report restructuring, and its specialized variant *Qwen2.5-Coder-7B* for code generation and analysis. They meet the above three criteria. We also adopt the recommended generation parameters [3] for the Qwen2.5-7B and Qwen2.5-Coder-7B models in our experiment. Our hybrid retrieval framework combines both sparse and dense retrieval methods to maximize relevance. The sparse retrieval component uses BM25 (k1 = 1.2, b = 0.75) for lexical matching, while the dense retrieval component employs an efficient nearest-neighbour search (i.e., 50 trees). All experiments were conducted on machines equipped with Nvidia RTX 3050 GPUs (8GB vRAM) in a local environment. On average, each bug report was processed in ≈4-5 minutes, with a range between 2 and 7 minutes, depending on the complexity of the report and size of the codebase.

4 Experiment

We curate a benchmark dataset of 106 bugs and evaluate our technique – RepGen – for automated bug reproduction using standard metrics [24]. To contextualize our work, we compare our technique against eight state-of-the-art LLMs of varying sizes. We also analyze the impact of various components of our approach through an ablation study and investigate the cases for which our approach fails. Finally, we validate the effectiveness of our generated code by involving 27 real developers and AI engineers. In our experiments, we thus answer four research questions as follows:

RQ1 - Effectiveness: How effectively does RepGen reproduce deep learning bugs compared to state-of-the-art LLMs?

RQ2 - Ablation Study: How different components of RepGen contribute to its bug reproduction capability?

RQ3 - Failure Analysis: What are the current limitations of RepGen, and does increasing model size improve RepGen's ability to reproduce deep learning bugs?

RQ4 - Usefulness: Do developers find our generated code practically useful for bug reproduction?

4.1 Dataset Construction

To construct the dataset for our experiment, we followed a systematic approach inspired by prior works [50]. We select GitHub, the largest resource for open-source repositories, and use its GitHub Search API [2] for repository selection. We first limit our search to Python repositories due to its frequent use in DL systems [5]. Next, we identify the repositories built on three frameworks: TensorFlow, Keras, and PyTorch, the most widely used DL frameworks [4]. We collected 223,485 repositories from GitHub after this step. Then, we make sure that each repository has at least one push to the main branch after January 2023, indicating their recent maintenance [50]. After this filtering, we retained 83,649 repositories for the subsequent analysis. Finally, we select such repositories that have a minimum of 10 watchers and 500 stars, a criterion used in existing literature [56] to exclude obsolete or low-quality repositories. After applying these criteria, we collected **653** repositories from GitHub. After the preliminary filtering above, we removed repositories with fewer than 10 closed bugs, ensuring that the selected repositories have a sufficient history of reported and resolved bugs. After this filtering step, we retained **392** repositories for manual analysis. Since our primary focus was on bugs in DL systems, we manually analyzed the 392 repositories and excluded non-DL, toy or educational repositories, leading to a total of *54* repositories for subsequent analysis. We then collect all the bugs that were resolved from each of the 54 repositories in the last two years (April 1 2023 - April 1 2025). We also carefully examined each bug report, fix commits, and necessary instructions and attempted to reproduce the bug in an isolated environment. If we could reproduce the bug within 1 hour, we would include this bug in the dataset. Bugs which could not be reproduced within an hour were marked as non-reproducible and excluded from further analysis. Through this process, we identified **106** reproducible bugs from **16** distinct repositories. We spent ≈190 person-hours on dataset construction.

4.2 Verification of Bug Reproduction

To verify whether our generated code accurately reproduces reported deep learning bugs, we employ a systematic approach inspired by prior work [63]. For *explicit bugs* [63], we confirm the bug reproduction by matching: (a) the error type (e.g., ValueError, CUDA OOM), (b) key diagnostic information (e.g., tensor shape mismatches), and (c) the execution context (e.g., training vs. inference phase). We execute our code five times with different random seeds and match their output with that of the bug report to confirm a successful reproduction. For *silent bugs* [67], we also execute the generated code five times with different random seeds to account for training non-determinism. The mean evaluation metrics (accuracy, loss, memory usage) from these runs are then compared against the values reported in the bug report. We consider a bug successfully reproduced if the mean metric falls within 5% error margin of the reported value, following established practices from

prior work [12, 57, 63]. We also verify the behavioural equivalence by examining whether the output of the generated code demonstrates the same failure patterns (e.g., NaN loss, excessive memory usage or performance degradation) as mentioned in the bug report.

4.3 Baseline Selection

To rigorously validate our approach, RepGen, we systematically chose eight open-source and closed-source LLMs as our baseline methods. We chose Llama-3 (8B, 70B), which are open-source, decoder-only transformer models with strong instruction-following capabilities and widely adopted by the research community. We also included DeepSeek-R1 (7B, 685B) and Qwen3 (8B) for their advanced reasoning capabilities. Furthermore, we included Qwen2.5-7B, as it was the base model for RepGen. Finally, we chose GPT-4.1, a highly capable proprietary model with its long context window and advanced code generation abilities. We employed three distinct prompting techniques for every LLM (zero-shot, few-shot, and chain-of-thought), and used DSPy to optimize our prompts. For smaller models (up to 8B), we conducted experiments on a local machine using Ollama, while for larger models (70B and above), we leveraged their respective APIs [6–8]. We generated the code using the recommended generation parameters (e.g., temperature, top_p, top_k) for the local models. To mitigate the effects of LLM non-determinism, we generated the code five times and evaluated each generation following our verification protocol (Section 4.2).

During our baseline selection, we also identified a few existing techniques. We observed that LIBRO failed to reproduce any DL bugs because it relies on high-quality, project-specific information. Other methods such as Otter(++) [11] and AEGIS [74] were also excluded as they do not provide replication packages, making their evaluation infeasible. Similarly, as discussed earlier (Section 1), GUI-based methods do not apply to our case as well. Thus, we did not select existing bug reproduction methods for comparison due to their inapplicability to our problem scenario.

4.4 Developer Study

To assess the benefits of RepGen in a real-life setting, we conduct a user study involving 27 software developers and AI engineers. We discuss our study setup, including questionnaire preparation, study design, and participant selection, as follows:

Introduction: In our questionnaire, we first provide necessary background context about the reproducibility of deep learning bugs and discuss the purpose of our study. Next, we provide a set of instructions and outline the structure of the subsequent sections.

Demographic Information: After the introduction, we gather a participant's background data through multiple-choice and numeric input questions. This includes years of professional software development experience, DL experience, and familiarity with frameworks like TensorFlow, PyTorch, Keras, and JAX. We also collect information about participants' prior experience debugging deep learning systems and the types of issues they commonly encounter.

Bug Reproduction: Each participant receives two bug reproduction tasks with varying difficulty levels. For each bug, participants in the experimental group received the bug report along with the generated code from RepGen, while the control group received only the bug report; the order of the reproduction tasks was randomized for each participant to prevent any ordering effects. Participants document their reproduction process, including root cause analysis, time spent, and detailed steps taken. We measure their success in bug reproduction through yes/no responses and their execution logs (if provided). We also collect qualitative data about their followed approach through open-ended questions.

Workload Assessment: Using NASA TLX [29] dimensions, we evaluate participants' experience through 5-point Likert scales that measure their mental demand, temporal demand, performance, effort, and frustration. This standardized assessment helps us quantify their cognitive load during bug reproduction.

Code Evaluation: The final section of our study focuses on evaluating the utility of our generated code. Through a combination of Likert scales and multiple-choice questions, participants rate the overall usefulness of the code and identify its valuable aspects for reproducibility of the bug, such as error handling, data preparation, and model configuration. Open-ended questions capture suggestions for further improvement or any complementary information.

Participant Selection: We first conducted a pilot study with two researchers and two developers to validate our questionnaire design. Based on their feedback, we refined the questionnaire by clarifying the ambiguous questions. We then recruited 27 professional developers and AI engineers through direct correspondence, organizational mailing lists, and professional networks. Participants were randomly assigned to either the experimental group (n = 14) or the control group (n = 13). The experimental group, which received detailed reproduction code, had a median experience of 5 years in software development and 2 years in deep learning development. Similarly, the control group also had a median of 5 years in software development and 2 years in deep learning. Both groups reported similar debugging experiences and framework expertise, with 92.57% having prior experience in debugging DL systems, and 100% having experience with at least one of the DL frameworks. Thus, the two groups possess equivalent relevant experience, mitigating any confounding factors in our study.

4.5 Metrics

To assess the effectiveness of our technique in bug reproduction, we employ the following metrics, as per the existing literature [24, 83]
Success Rate (SR) measures the percentage of deep learning bugs reproduced by a technique as follows.

$$SR = \frac{\text{Number of successfully reproduced bugs}}{\text{Total number of bugs in the dataset}} \times 100\%$$

Time to Reproduce (TTR) captures the time required to successfully reproduce a bug, measured from the start of the reproduction attempt until the bug is confirmed.

4.6 Results

4.6.1 ***Answering RQ$_1$*** *– Effectiveness.* Table 1 shows the effectiveness (e.g., success rate) of RepGen and eight other baseline techniques in reproducing deep learning bugs. Our technique, RepGen, achieved an 80.19% success rate in bug reproduction, surpassing the other baselines powered by various LLMs. While prompting has the potential to enhance an LLM's built-in reasoning abilities, it might fall short in reproducing DL bugs as follows.

Table 1: Effectiveness of RepGen for DL Bug Reproduction

Model	Zero-Shot	Few-Shot	CoT
RepGen		**80.19% ±1.51%**	
Llama-3-8B	18.87% ±3.29%	33.02% ±2.82%	28.30% ±2.67%
Llama-3-70B	44.34% ±2.15%	17.92% ±2.57%	17.92% ±1.41%
DeepSeek-R1-7B	36.79% ±3.66%	40.57% ±2.56%	22.64% ±3.66%
DeepSeek-R1-685B	49.06% ±1.92%	60.38% ±1.94%	42.45% ±1.64%
Qwen3-8B	23.58% ±3.07%	30.19% ±2.56%	34.91% ±3.25%
GPT-4.1	42.45% ±1.51%	53.77% ±1.62%	40.57% ±0.75%
Qwen2.5-7B	21.70% ±2.39%	28.30% ±3.61%	30.19% ±2.06%
Qwen2.5-Coder-7B	25.47% ±3.78%	33.02% ±2.92%	28.30% ±1.94%

Note: RepGen does not use any in-context learning or CoT reasoning.

First, zero-shot prompting [42] allows LLMs to respond based on their pre-trained knowledge. They cannot access relevant codebase information, which is critical for reproducing DL bugs. As shown by Llama-3-8B's 18.87% and Qwen2.5-7B's 21.64% reproduction rates, zero-shot prompting results in poor performance. Even larger models, such as Llama-3-70B and GPT-4.1, struggle significantly and deliver a maximum of 49% success rate. All these findings suggest that zero-shot prompting might not be sufficient for reproducing DL bugs. It cannot integrate project-specific context, limiting the capabilities of the LLMs. On the other hand, RepGen solves such a knowledge gap by constructing a learning-enhanced context and equipping the LLM agent with appropriate information (e.g., training pipelines, dependencies) for code generation.

Second, few-shot prompting [18] provides an LLM with a small set of example inputs and outputs to show desired behaviours and to guide the LLM's response. However, its usefulness during DL bug reproduction might be limited. While some models, like Qwen2.5-Coder-7B, show a small improvement to 33.02% from their zero-shot performance, other models, such as Llama-3-70B, surprisingly perform worse, degrading to 17.92% compared to their zero-shot performance. DeepSeek-R1-685B, improves significantly and reaches to 60.38% with few-shot learning, but this is still lower than RepGen's performance. Furthermore, as shown in the Venn diagram in Fig. 5, RepGen reproduced 35 unique bugs that DeepSeek-R1 + Few-Shot could not reproduce. Thus, our findings suggest that a limited set of examples cannot possibly overcome all the challenges of DL bugs, such as the variety of bug types, non-determinism, architectural setups, and environmental settings. Furthermore, few-shot prompting is restricted by the fixed limits of the model's context window. On the other hand, RepGen overcomes these limitations by providing a dynamically constructed, enriched context tailored to specific bugs, which helps our agent, powered by the LLM, to understand the problem better and generate code capable of reproducing bugs.

Finally, chain of thought (CoT) [77] prompting encourages LLMs to explain their intermediate reasoning steps before generating a final response. While helpful for general-purpose tasks, its use did not result in a high performance when generating code capable of reproducing deep learning bugs. Models like Llama-3-70B (17.92%) and DeepSeek-R1-7B (22.64%) perform worse with CoT than with zero-shot prompting. Even where small gains are seen (e.g., Qwen3-8B), the overall success in bug reproduction is still low (34.91%). Thus, the reasoning steps of LLMs based on incomplete information or only pre-trained knowledge might not lead to an appropriate plan for a complex problem scenario like ours. More importantly,

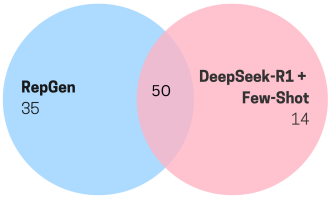

Figure 5: RepGen v DeepSeek-R1

CoT has no built-in way to check if the generated code is syntactically correct, shares symptoms with a bug report, or demonstrates the erroneous runtime behaviour. On the other hand, our plan is generated by our LLM based on its reasoning capabilities, the refined bug report and the learning-enhanced context. Furthermore, we employ a multi-stage feedback mechanism, with an iterative cycle of generating, validating, and refining, which leads to code capable of reproducing the DL bugs.

We also investigated whether integrating RepGen's artifacts enhances the reproduction capabilities of the best-performing LLM (DeepSeek-R1-685B). Using RepGen's plan alone and context alone, DeepSeek-R1-685B reproduced 63.21% and 68.87% of bugs, respectively. Combining the learning-enhanced context with the plan increased this to 71.70%. Similarly, when DeepSeek-R1-685B was provided a restructured bug report along with the context and plan, it achieved 65.04%. This performance further improved to 72.64% when run with an agentic setup that enhances the context, plan, and restructured report with lightweight PyLint and AST feedback.

Statistical Significance Tests: To demonstrate RepGen's effectiveness, we conducted paired comparisons between RepGen and each of the 24 baseline configurations using McNemar's test on the 2×2 contingency tables for the 106 shared bugs. McNemar's test [49] is appropriate for paired binary outcomes and for assessing whether the proportion of bugs reproduced by RepGen differs significantly from each baseline counterpart. For each comparison, we report the p-values and the Haldane–Anscombe adjusted odds ratio (hereafter, "odds ratio") [28]. To account for multiple comparisons, we applied the Benjamini–Hochberg correction [15] to these tests, yielding BH-adjusted p-values (q-values). All 24 comparisons remained statistically significant after BH correction ($q \leq 3.80 \times 10^{-3}$), with odds ratios ranging from 2.45 to 19.86. For example, when comparing RepGen to DeepSeek-R1-685B (few-shot), there were $b = 35$ cases where RepGen succeeded and the baseline failed, versus $c = 14$ cases in the opposite direction, giving an odds ratio of 2.45 and a BH-adjusted q-value of 3.80×10^{-3}. We also calculated 95% confidence intervals for the difference in mean success rates for bug reproduction over five runs between RepGen and the baseline using the Newcombe hybrid score method [55], finding that the difference is 19.8% with a 95% CI of 7.0%–29.7%, indicating that RepGen reproduces between 7% and 30% more of the shared bugs than DeepSeek-R1-685B. Note that this statistic differs from the odds ratio, which summarizes the relative odds of reproduction. Overall, these results demonstrate that RepGen outperforms all baselines in bug reproduction with statistical significance.

4.6.2 ***Answering RQ$_2$*** *– Ablation Study.* Table 2 shows the results of our ablation study. RepGen has six major components across different steps of its workflow. We disable each of them and assess

Table 2: Impact of Components on Bug Reproduction Success

Config.	Components						Success Rate
	R1	R2	R3	R4	R5	R6	
No Relevance F.	●	●	●	●		●	17.92% ± 1.62%
No Planning	●		●	●	●	●	21.70% ± 2.18%
No Restructuring		●	●	●	●	●	22.64% ± 2.18%
No Compilation F.	●	●		●	●	●	33.96% ± 2.01%
No Static Analysis F.	●	●	●		●	●	41.51% ± 2.15%
No Runtime F.	●	●	●	●	●		46.23% ± 1.62%
Complete System	●	●	●	●	●	●	80.19% ± 1.33%

R1=BR Restructuring, **R2**=Planning, **R3**=Compilation F., **R4**=Static Analysis F., **R5**=Relevance F., **R6**=Runtime F., ●=Component Present

Table 3: Impact of Retrieval Components on Reproduction

Config.	Components							Success Rate
	C1	C2	C3	C4	C5	C6	C7	
No ANN		●	●	●	●	●	●	26.42% ± 2.83%
No BM25	●		●	●	●	●	●	24.53% ± 1.89%
No Dependency Ext.	●	●		●	●	●	●	34.91% ± 0.94%
No Module Part.	●	●	●		●	●	●	23.58% ± 2.45%
No Reranker	●	●	●	●		●	●	30.19% ± 1.32%
No Training Loop Ext.	●	●	●	●	●		●	28.30% ± 2.11%
No Training Loop Rank.	●	●	●	●	●	●		20.75% ± 1.63%
Complete System	●	●	●	●	●	●	●	80.19% ± 1.33%

C1=ANN, **C2**=BM25, **C3**=Dependency Extraction, **C4**=Module Partitioning, **C5**=Reranker, **C6**=Training Loop Extraction, **C7**=Training Loop Ranking

RepGen's code generation capability towards DL bug reproduction as follows. First, we see that the removal of the relevance-checking component has the most significant impact, with performance dropping to 17.92%. The relevance estimation between code and the bug report helps RepGen discard noisy or irrelevant code and trigger a re-generation of code. Second, the removal of the planning component reduces the success rate in bug reproduction to 21.70%. Such a decrease indicates that breaking down bug reproduction tasks into smaller, manageable sub-tasks (a.k.a., planning) equips the LLM with a structured way to generate appropriate code and reproduce bugs. Third, the removal of the bug report restructuring component also has a major impact on RepGen and reduces the success rates to 22.64%. This impact indicates that the restructuring of a bug report enhances its clarity and precision and thereby improves the LLM's ability to understand and reproduce issues. Fourth, the removal of compilation checks in RepGen reduced the success rate in bug reproduction to 33.96%, underscoring the importance of syntactic correctness in generated code. Finally, the removal of static analysis feedback and runtime behaviour approximation had significant impacts, reducing success rates in bug reproduction to 46.23% and 41.51%, respectively. These components serve as important verification steps, ensuring that the code has relevant symptoms and is likely to reproduce the reported bugs.

We also analyze the impact of the seven components used in our context construction step (Step 1a, Fig. 3), with results summarized in Table 3. Removal of the training loop ranking component caused the largest performance drop, reducing the success rate to 20.75%. This underscores its critical role in identifying the training loop most relevant to the reported bug, ensuring that the constructed context focuses on the correct portion of the codebase. Similarly, omitting module partitioning reduced the success rate to 23.58%, highlighting that prioritizing modules (e.g., components or classes) based on the relevance of the code snippets rather than the sheer number of snippets is essential for building an informative context. Our hybrid retrieval mechanism also proved vital for context construction. Disabling BM25 search, which captures exact lexical matches, reduced RepGen's success rate to 24.53%, while removing ANN search, responsible for retrieving semantically related code, resulted in a 26.42% success rate. These drops demonstrate that both lexical and semantic retrieval are complementary and necessary for constructing a comprehensive context. Removal of the training loop extraction component lowered RepGen's performance to 28.30%, indicating that incomplete loops without core training logic can weaken the context. Removing reranking, which refines the relevance of retrieved snippets, decreased success to 30.19%, showing that careful prioritization of candidate code is important for context quality. Finally, omitting dependency extraction, responsible for capturing imports, helper functions, and referenced variables, reduced RepGen's success rate to 34.91%, highlighting the need for a complete and executable context. These findings collectively show that each element of our context construction pipeline is vital for the high performance of RepGen. Our findings above align with earlier work [63], suggesting that successful bug reproduction requires a multi-faceted approach. The strong performance of RepGen (80.19%) compared to its ablated versions also suggests that each component addresses a specific challenge and contributes significantly when reproducing DL bugs.

4.6.3 ***Answering RQ$_3$*** *– Failure Analysis.* To understand difficulty of reproducing certain deep learning bugs, we manually analyzed the 21 bugs that RepGen failed to reproduce. Two researchers, the lead author and an independent collaborator, each with over five years of experience in deep learning and software engineering, labelled these bugs using multiple established DL bug taxonomies [34, 35, 51] and identified their bug types, symptoms, and root causes. We also examined each non-reproduced bug's report and associated developer discussion to extract its symptoms and root causes, and mapped them to the closest taxonomy-defined type. The inter-rater agreement, measured using Cohen's kappa was 0.849, which indicates an almost perfect agreement [48]. All disagreements were subsequently resolved through discussion between the two researchers. Based on the labelled types, we then grouped the bugs into three higher-level categories: (a) API/Dependency bugs (10/21), (b) environment-dependent bugs (5/21) and (c) data-dependent bugs (6/21), as shown in Table 4. Reproducing these categories of bugs is challenging primarily due to their reliance on specific external conditions. For example, RepGen struggles with newer APIs and updates since they were released after the knowledge cutoff of the base model, Qwen2.5-Coder-7B. Environment-dependent bugs are also difficult to reproduce because of complex distributed systems, resource allocation across multiple GPUs, and precise hardware requirements. Similarly, data-dependent bugs are challenging due to reliance on specific datasets, file system structures, and particular data formats and paths. Moreover, we also found that each failure type aligns with distinctive pipeline signals in RepGen. API or dependency mismatches result in low BM25 or ANN relevance scores and unresolved dependency graphs. Environment-dependent bugs exhibit incomplete context coverage or unresolvable hardware-specific requirements, as indicated by runtime feedback showing that the bug cannot be reproduced in the environment created by

Table 4: Non-Reproducible Bug Categories

Category	Definition	Example
API/Dependency Bug	An error caused by an incompatibility between the code and its dependencies (e.g., libraries), often due to version changes.	A model fails during inference with a `TypeError` because the code passes an argument (`'end_token_id'`) to a function from the KerasNLP library that no longer accepts it in its updated version.
Environment-Dependent Bug	A bug that appears in a specific hardware or software configuration, such as a multi-GPU or distributed computing setup.	A training job using TensorFlow's `multi_worker_mirrored` strategy on a two-node cluster fails to show any performance increase, indicating a scaling problem specific to the distributed environment.
Data-Dependent Bug	An error caused by input data issues, such as missing files, incorrect paths, or improper formatting.	A model training script crashes during the first epoch with a `NotFoundError` because it cannot locate a file (`17e00d.jpg`) that is listed in the dataset but is not present at the path.

the generated code. Data-dependent bugs produce early failures in static analysis or at runtime due to missing datasets or paths. These consistent signals explain why some bugs cannot be reproduced and provide future directions for improving RepGen's workflow.

Experiment with GPT-4.1: Given that our technique, empowered by Qwen2.5-Coder-7B, reproduced 85 out of 106 bugs (80.19%), we further investigated whether a larger model could reproduce the 21 remaining bugs. We thus employed GPT-4.1, one of the most capable models, and conducted experiments using these bugs. RepGen, empowered with GPT-4.1, reproduced 8 out of the 21 bugs (38.10%), demonstrating that larger models can tackle some of the challenging cases. This improvement is primarily due to GPT-4.1's enhanced ability to identify and resolve API or dependency mismatches, reproducing 5 of 10 such bugs (50.0% ± 15.8%). For instance, GPT-4.1 successfully reproduced `pip install` failures caused by invalid resolver options or `ImportError` issues stemming from incompatible Keras versions, reflecting a stronger understanding of evolving library ecosystems. Additionally, GPT-4.1's improved code analysis enabled the reproduction of two environment-dependent bugs (40.0% ± 21.9%) where resolution involved code modifications rather than complex infrastructure changes, such as correcting tensor dimension mismatches during training. Data-dependent bugs, such as missing image files, were largely unreproducible (16.7% ± 15.2%) because automated techniques cannot create or manage external file system content. Conversely, RepGen, even when powered by GPT-4.1, failed to reproduce the remaining 13 bugs due to their dependence on highly specific external conditions [37]. Many of these were environment-dependent bugs requiring control over distributed systems, hardware configurations, or intricate framework interactions that exceed the capabilities of current LLMs. Examples include multi-node scaling issues in TensorFlow and `ModuleNotFoundError` for specialized libraries (e.g., DeepSpeed).

Given the evidence above, model size is a factor, but not the sole determinant of success in reproducing DL bugs. This finding reinforces that the primary challenge of the automated DL bug reproduction is the lack of awareness of the bug's external context. While a larger model can resolve more issues based on its broader knowledge, it still fails when the reproduction requires precise environment-related context that it cannot access. Thus, future approaches should focus on improving the quality of the context provided to LLM agents, rather than relying on increased model sizes.

4.6.4 ***Answering RQ$_4$*** *– Usefulness.* To evaluate the usefulness of our generated code in a practical setting, we conducted a controlled study involving 27 professional developers and AI engineers (n=14 experimental, n=13 control). To ensure that our developer study had sufficient statistical power to detect meaningful differences between two experimental settings, we conducted an a priori power analysis. This analysis estimates whether the planned sample size can reliably detect effects of practical significance. The results indicate that our study is adequately powered (80% power, α=0.05) to detect large effects (Cliff's $\delta \geq 0.47$) in continuous measures (e.g., time) and substantial differences in success rates. This aligns with typical effect sizes observed in software engineering studies on tool evaluation and is appropriate for detecting meaningful, practical improvements in bug reproduction [23, 39]. To analyze the results, we employed the same strategy as in prior work [63].

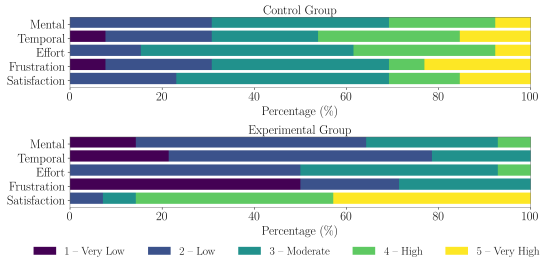

Figure 6: Developer Study Results

Each participant was assigned two bug reports discussing deep learning bugs from our dataset. The control group (i.e, did not receive our generated code) achieved a 73.07% success rate in reproducing bugs, whereas the experimental group (i.e., received our generated code) achieved a 96.42% success rate, indicating an improvement of **23.35% ± 18.39%**, which show that our generated code substantially aids developers in reproducing deep learning bugs. The analysis of participants' reported reproduction process revealed that the experimental group was able to understand the bugs better. For example, one participant explained their understanding as follows: *"Shape mismatch during commit loss calculation due to incorrect tensor reshaping when mask is applied"*. On the other hand, control group participants often expressed uncertainty: *"Didn't understand how masking affects training"*. Our provided code also helped reduce the average time to reproduce by **56.8% ± 16.8%**, from 25.73 minutes for the control group to 11.11 minutes for the experimental group. For example, one participant from the experimental group explained their reproduction approach as follows: *"Ran provided code, confirmed TF version, observed expected error"*. On the other hand, a participant from the control group faced challenges: *"Tried multiple inputs; all failed due to unrelated tensor shape issues"*. Further details about the participants' experiences are available in our replication package [9].

To determine whether the presence of our generated code (i.e., categorical variable) significantly influenced the bug reproduction success of the two groups (i.e, categorical variable), we performed a χ^2 test of independence. We found that there was a statistically significant difference (p = 0.0423, ϕ = 0.276) in success rates when

the code was present or absent. We also performed a Mann-Whitney U test to assess the difference in the time taken by the participants from the two groups, and found out that there is a significant difference ($p = 0.000$, Cliff's $\delta = 0.84$). Thus, the above findings suggest a significant impact of RepGen in a practical setting on the success rate and time taken to reproduce deep learning bugs.

We also assessed participants' cognitive load using the NASA Task Load Index (NASA-TLX). As shown in Fig. 6, the experimental group reported significantly lower cognitive burden across all dimensions, including a 43.2% ± 28.2% reduction in frustration and a 25.6% ± 23.1% reduction in mental demand. Furthermore, participants in the experimental group also reported higher performance (4.21 vs. 3.23), corresponding to a 30.34% ± 24.46% improvement relative to the control group. Based on our collected responses, the experimental group also rated the overall usefulness of our generated code at 4.57 out of 5, with 92.30% rating it as "Useful" or "Highly Useful" (>4). All participants in the experimental group would also prefer the inclusion of the generated code in bug reports. Thus, all the findings above suggest that RepGen's generated code offers significant benefits by improving bug-reproduction success rates, reducing time spent, and lowering developers' cognitive load.

5 Threats To Validity

Threats to *internal validity* relate to experimental errors and biases. Replication of baseline techniques could pose a threat. To mitigate this, we used the official versions of the eight models and employed three widely used prompting techniques. We further mitigated bias by repeating our experiments five times and comparing performance across these trials [13]. Threats to *construct validity* concern the rigour of our evaluation methodology [65]. During dataset construction, we implemented a stringent verification protocol involving the replication of operating environment, error manifestation, and time spent for reproduction (e.g., one-hour) to simulate real-world constraints [63]. For silent bugs, we accepted the metrics that fall within 5% margin of reported values, following established practices from prior work [57, 63]. Thus, our systematic approach ensured that only verifiably reproducible bugs were included. Another threat could arise from the evaluation metrics. To mitigate this threat, we used the evaluation metrics from the existing literature [24, 63]. Thus, the threats to construct validity might be mitigated. Threats to *external validity* relate to generalizability [25]. Our dataset contains bugs from 16 actively maintained projects spanning three frameworks, with a median of 8,282 stars and 258K LOC, indicating their strong real-world relevance. Our Python-centric implementation (e.g., PyLint integration) may warrant adaptation for non-Python DL ecosystems. However, this threat is mitigated by RepGen's modular design, which permits framework-specific adaptations. We have also released our replication package [9] to support the reproducibility of our findings.

6 Related Work

Localization and repair of deep learning bugs: Over the last two decades, several techniques have been proposed to localize and repair bugs in deep learning (DL) systems [19, 30, 36, 76, 80]. For example, DeepLocalize [76] uses dynamic analysis (e.g., monitoring of model parameters) to detect and localize faults in neural network training, while DeepFD [19] applies runtime feature analysis (e.g., gradients, weights) to classify and locate faults. AutoTrainer [80] monitors DL training to detect issues like vanishing gradients and applies built-in fixes. More recently, Harzevili et al. [30] study input validation faults (a.k.a, checker bugs) in PyTorch and TensorFlow and leverage retrieval-augmented LLM prompts to generate their fixes. However, these methods assume the bug has already been reproduced and do not automate the reproduction of DL bugs. In contrast, we focus on automating DL bug reproduction to support downstream tasks such as fault diagnosis and repair.

Automated bug reproduction: Traditional approaches for bug reproduction employ GUI-based testing, program analysis, and natural language processing. For example, ReCDroid+ [83] synthesizes replayable event sequences from Android issue reports, whereas recent techniques like ReBL [24] and AdbGPT [73] use LLMs to extract and execute reproduction steps. These methods typically assume deterministic execution and well-defined oracles such as crashes or exceptions. Unfortunately, these techniques are not suitable for DL systems due to their non-determinism (e.g., random initialization) and complex, silent failure modes (e.g., accuracy degradation) [34, 63]. A recent work, AEGIS [74], automates bug reproduction using LLM agents and external tools but overlooks DL-specific challenges such as non-determinism, hardware and dataset dependencies, weak oracles, and incomplete issue descriptions, limiting reliability. Similarly, Otter++ [11] retrieves generic code and generates test cases. In contrast, RepGen addresses these limitations via three key innovations: it constructs a DL-specific learning-enhanced context; integrates runtime-feedback-driven validation to estimate reproduction success; and generates complete, executable code for DL bug reproduction.

7 Conclusion & Future Work

Deep learning bugs are difficult to reproduce because of the inherent nondeterminism of DL models and their tight coupling with specific hardware (e.g., GPU) and software environments (e.g., underlying frameworks). In this paper, we present *RepGen*, a novel approach for automatically reproducing deep learning bugs. Our technique accelerates and systematizes the reproduction of deep learning bugs – by integrating a learning-enhanced context, planning and a generate-validate-refine loop with multiple verification mechanisms. Evaluation on 106 real-world bugs demonstrated RepGen's superior performance, achieving an 80.19% reproduction rate — 19.81% higher than the best baseline. Furthermore, a controlled developer study confirmed the practical usefulness of RepGen by improving the reproduction success rates by 23.35%, reducing average time to reproduce by 56.8%, and substantially lowering developers' cognitive load. Future work includes extending support for distributed training bugs and integrating with existing tools localizing or repairing deep learning bugs. Thus, by enabling reliable bug reproduction, *RepGen* advances the reliability and trustworthiness of DL systems.

Acknowledgments: This work was supported by the Natural Sciences and Engineering Research Council of Canada (Discovery Grant RGPIN-03236), the Fonds de recherche du Québec (FRQ), and the Canadian Institute for Advanced Research (CIFAR).

References

[1] 2017. Cosine Distance, Cosine Similarity, Angular Cosine Distance, Angular Cosine Similarity. https://www.itl.nist.gov/div898/software/dataplot/refman2/auxillar/cosdist.htm Accessed: 2025-07-07.

[2] 2022. GitHub REST API Documentation. https://docs-internal.github.com/en/rest?apiVersion=2022-11-28 Accessed: 2025-07-02.

[3] 2024. Generation Configuration for Qwen2.5-Coder-32B-Instruct. https://huggingface.co/Qwen/Qwen2.5-Coder-32B-Instruct/blob/main/generation_config.json Accessed: 2025-07-12.

[4] 2024. ML Engineer comparison of Pytorch, TensorFlow, JAX, and Flax. https://softwaremill.com/ml-engineer-comparison-of-pytorch-tensorflow-jax-and-flax/ Accessed: 2025-06-28.

[5] 2024. What's the best programming language for machine learning? https://www.techtarget.com/searchenterpriseai/tip/Whats-the-best-programming-language-for-machine-learning Accessed: 2025-07-04.

[6] 2025. DeepSeek API Documentation. https://api-docs.deepseek.com/ Accessed: 2025-07-17.

[7] 2025. GroqCloud. https://console.groq.com Accessed: 2025-07-17.

[8] 2025. OpenAI API. https://openai.com/api/ Accessed: 2025-07-17.

[9] 2025. Replication Package for RepGen. https://github.com/mehilshah/ICSE26-RepGen

[10] Peter Martey Addo, Dominique Guegan, and Bertrand Hassani. 2018. Credit risk analysis using machine and deep learning models. *Risks* 6, 2 (2018), 38.

[11] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. 2025. Otter: Generating Tests from Issues to Validate SWE Patches. *arXiv preprint arXiv:2502.05368* (2025).

[12] Saeed S Alahmari, Dmitry B Goldgof, Peter R Mouton, and Lawrence O Hall. 2020. Challenges for the repeatability of deep learning models. *IEEE Access* 8 (2020), 211860–211868.

[13] Andrea Arcuri and Lionel Briand. 2014. A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability* 24, 3 (2014), 219–250.

[14] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. 2016. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268* (2016).

[15] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal statistical society: series B (Methodological)* 57, 1 (1995), 289–300.

[16] Daniel S Berman, Anna L Buczak, Jeffrey S Chavis, and Cherita L Corbett. 2019. A survey of deep learning methods for cyber security. *Information* 10, 4 (2019), 122.

[17] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2024. Repairagent: An autonomous, llm-based agent for program repair. *arXiv preprint arXiv:2403.17134* (2024).

[18] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[19] Jialun Cao, Meiziniu Li, Xiao Chen, Ming Wen, Yongqiang Tian, Bo Wu, and Shing-Chi Cheung. 2022. Deepfd: Automated fault diagnosis and localization for deep learning programs. In *Proceedings of the 44th international conference on software engineering*. 573–585.

[20] Partha Chakraborty, Mahmoud Alfadel, and Meiyappan Nagappan. 2024. Rlocator: Reinforcement learning for bug localization. *IEEE Transactions on Software Engineering* (2024).

[21] Junkai Chen, Zhiyuan Pan, Xing Hu, Zhenhao Li, Ge Li, and Xin Xia. 2024. Reasoning Runtime Behavior of a Program with LLM: How Far Are We?. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 140–152.

[22] Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Denys Poshyvanyk, Massimiliano Di Penta, and Gabriele Bavota. 2021. An Empirical Study on the Usage of BERT Models for Code Completion. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 108–119. doi:10.1109/MSR52588.2021.00024

[23] Francisco Gomes de Oliveira Neto, Richard Torkar, Robert Feldt, Lucas Gren, Carlo A Furia, and Ziwei Huang. 2019. Evolution of statistical analysis in empirical software engineering research: Current state and steps forward. *Journal of Systems and Software* 156 (2019), 246–267.

[24] Sidong Feng and Chunyang Chen. 2024. Prompting is all you need: Automated android bug replay with large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[25] Michael G Findley, Kyosuke Kikuta, and Michael Denly. 2021. External validity. *Annual review of political science* 24, 1 (2021), 365–393.

[26] Grand View Research. 2024. Deep Learning Market Size & Share Report, 2024-2030. *Market Research Report* (2024). https://www.grandviewresearch.com/industry-analysis/deep-learning-market

[27] Sorin Grigorescu, Bogdan Trasnea, Tiberiu Cocias, and Gigel Macesanu. 2020. A survey of deep learning techniques for autonomous driving. *Journal of field robotics* 37, 3 (2020), 362–386.

[28] JB Haldane. 1956. The estimation and significance of the logarithm of a ratio of frequencies. *Annals of human genetics* 20, 4 (1956), 309–311.

[29] Sandra G Hart. 2006. NASA-task load index (NASA-TLX); 20 years later. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 50. Sage publications Sage CA: Los Angeles, CA, 904–908.

[30] Nima Shiri Harzevili, Mohammad Mahdi Mohajer, Jiho Shin, Moshi Wei, Gias Uddin, Jinqiu Yang, Junjie Wang, Song Wang, Zhen Ming, Jiang, and Nachiappan Nagappan. 2024. Checker Bug Detection and Repair in Deep Learning Libraries. arXiv:2410.06440 [cs.SE] https://arxiv.org/abs/2410.06440

[31] Nima Honarmand and Josep Torrellas. 2014. Replay debugging: Leveraging record and replay for program debugging. *ACM SIGARCH Computer Architecture News* 42, 3 (2014), 445–456.

[32] Kaifeng Huang, Bihuan Chen, Susheng Wu, Junming Cao, Lei Ma, and Xin Peng. 2023. Demystifying dependency bugs in deep learning stack. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 450–462.

[33] Yuchao Huang, Junjie Wang, Zhe Liu, Mingyang Li, Song Wang, Chunyang Chen, Yuanzhe Hu, and Qing Wang. 2025. One Sentence Can Kill the Bug: Auto-Replay Mobile App Crashes From One-Sentence Overviews. *IEEE Transactions on Software Engineering* (2025).

[34] Nargiz Humbatova, Gunel Jahangirova, Gabriele Bavota, Vincenzo Riccio, Andrea Stocco, and Paolo Tonella. 2020. Taxonomy of real faults in deep learning systems. In *Proceedings of the ACM/IEEE 42nd international conference on software engineering*. 1110–1121.

[35] Md Johirul Islam, Giang Nguyen, Rangeet Pan, and Hridesh Rajan. 2019. A Comprehensive Study on Deep Learning Bug Characteristics *(ESEC/FSE 2019)*. Association for Computing Machinery, New York, NY, USA, 510–520. doi:10.1145/3338906.3338955

[36] Sigma Jahan, Mehil B Shah, Parvez Mahbub, and Mohammad Masudur Rahman. 2025. Improved Detection and Diagnosis of Faults in Deep Neural Networks Using Hierarchical and Explainable Classification. *arXiv preprint arXiv:2501.12560* (2025).

[37] Sigma Jahan, Mehil B Shah, and Mohammad Masudur Rahman. 2024. Towards understanding the challenges of bug localization in deep learning systems. *arXiv preprint arXiv:2402.01021* (2024).

[38] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-Planning Code Generation with Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 33, 7, Article 182 (Sept. 2024), 30 pages. doi:10.1145/3672456

[39] Vigdis By Kampenes, Tore Dybå, Jo E Hannay, and Dag IK Sjøberg. 2007. A systematic review of effect size in software engineering experiments. *Information and Software Technology* 49, 11-12 (2007), 1073–1086.

[40] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.

[41] Anjan Karmakar and Romain Robbes. 2021. What do pre-trained code models know about code?. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1332–1336.

[42] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.

[43] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[44] Yi Li, Shaohua Wang, and Tien N Nguyen. 2022. Dear: A novel deep learning-based approach for automated program repair. In *Proceedings of the 44th international conference on software engineering*. 511–523.

[45] Parvez Mahbub and Mohammad Masudur Rahman. 2024. Predicting line-level defects by capturing code contexts with hierarchical transformers. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 308–319.

[46] Parvez Mahbub, Ohiduzzaman Shuvo, and Mohammad Masudur Rahman. 2023. Explaining software bugs leveraging code structures in neural machine translation. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 640–652.

[47] Junayed Mahmud, Nadeeshan De Silva, Safwat Ali Khan, Seyed Hooman Mostafavi, SM Hasan Mansur, Oscar Chaparro, Andrian Marcus, and Kevin Moran. 2024. On using gui interaction data to improve text retrieval-based bug localization. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.

[48] Mary L McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica* 22, 3 (2012), 276–282.

[49] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157.

[50] Mohammad Mehdi Morovati, Amin Nikanjam, Foutse Khomh, and Zhen Ming (Jack) Jiang. 2023. Bugs in Machine Learning-Based Systems: A Faultload Benchmark. *Empirical Softw. Engg.* 28, 3 (apr 2023), 33 pages. doi:10.1007/s10664-023-10291-1

[51] Mohammad Mehdi Morovati, Amin Nikanjam, Florian Tambon, Foutse Khomh, and Zhen Ming Jiang. 2024. Bug characterization in machine learning-based systems. *Empirical Software Engineering* 29, 1 (2024), 14.

[52] Prabhat Nagarajan, Garrett Warnell, and Peter Stone. 2018. The impact of nondeterminism on reproducibility in deep reinforcement learning. (2018).

[53] Mathieu Nayrolles, Abdelwahab Hamou-Lhadj, Sofiène Tahar, and Alf Larsson. 2015. JCHARMING: A bug reproduction approach using crash traces and directed model checking. In *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 101–110.

[54] MM Abid Naziri, Aman Kumar Singh, Benjamin Wu, Feiran Qin, Saikat Dutta, and Marcelo d'Amorim. 2025. BugsInDLLs: A Database of Reproducible Bugs in Deep Learning Libraries to Enable Systematic Evaluation of Testing Techniques. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 61–65.

[55] Robert G Newcombe. 1998. Improved confidence intervals for the difference between binomial proportions based on paired data. *Statistics in medicine* 17, 22 (1998), 2635–2650.

[56] Nafiseh Nikeghbal, Amir Hossein Kargaran, Abbas Heydarnoori, and Hinrich Schütze. 2023. Girt-data: Sampling github issue report templates. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 104–108.

[57] Hung Viet Pham, Shangshu Qian, Jiannan Wang, Thibaud Lutellier, Jonathan Rosenthal, Lin Tan, Yaoliang Yu, and Nachiappan Nagappan. 2020. Problems and opportunities in training deep learning software systems: An analysis of variance. In *Proceedings of the 35th IEEE/ACM international conference on automated software engineering*. 771–783.

[58] PwC. 2023. *AI Predictions Survey*. Technical Report. PricewaterhouseCoopers. https://www.pwc.com/us/en/tech-effect/ai-analytics/ai-predictions.html

[59] Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[60] Ripon K Saha, Matthew Lease, Sarfraz Khurshid, and Dewayne E Perry. 2013. Improving bug localization using structured information retrieval. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 345–355.

[61] Asif Mohammed Samir and Mohammad Masudur Rahman. 2025. Improved IR-based Bug Localization with Intelligent Relevance Feedback. arXiv:2501.10542 [cs.SE] https://arxiv.org/abs/2501.10542

[62] Mehil B Shah, Mohammad Masudur Rahman, and Foutse Khomh. 2024. Towards Understanding the Impact of Data Bugs on Deep Learning Models in Software Engineering. *arXiv preprint arXiv:2411.12137* (2024).

[63] Mehil B Shah, Mohammad Masudur Rahman, and Foutse Khomh. 2025. Towards enhancing the reproducibility of deep learning bugs: an empirical study. *Empirical Software Engineering* 30, 1 (2025), 23.

[64] Dinggang Shen, Guorong Wu, and Heung-Il Suk. 2017. Deep learning in medical image analysis. *Annual review of biomedical engineering* 19 (2017), 221–248.

[65] Gregory T Smith. 2005. On construct validity: issues of method and measurement. *Psychological assessment* 17, 4 (2005), 396.

[66] Mozhan Soltani, Pouria Derakhshanfar, Annibale Panichella, Xavier Devroey, Andy Zaidman, and Arie van Deursen. 2018. Single-objective versus multi-objectivized optimization for evolutionary crash reproduction. In *Search-Based Software Engineering: 10th International Symposium, SSBSE 2018, Montpellier, France, September 8-9, 2018, Proceedings 10*. Springer, 325–340.

[67] Florian Tambon, Amin Nikanjam, Le An, Foutse Khomh, and Giuliano Antoniol. 2024. Silent bugs in deep learning frameworks: an empirical study of keras and tensorflow. *Empirical Software Engineering* 29, 1 (2024), 10.

[68] Yuriy Tymchuk, Mohammad Ghafari, and Oscar Nierstrasz. 2018. JIT feedback: What experienced developers like about static analysis. In *Proceedings of the 26th Conference on Program Comprehension*. 64–73.

[69] Daisuke Wakabayashi. 2018. Self-driving uber car kills pedestrian in Arizona, where Robots Roam. https://www.nytimes.com/2018/03/19/technology/uber-driverless-fatality.html Accessed on December 17, 2023.

[70] Chenxu Wang, Tianming Liu, Yanjie Zhao, Minghui Yang, and Haoyu Wang. 2025. LLMDroid: Enhancing Automated Mobile App GUI Testing Coverage with Large Language Model Guidance. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1001–1022.

[71] Di Wang, Matthias Galster, and Miguel Morales-Trujillo. 2024. A systematic mapping study of bug reproduction and localization. *Information and Software Technology* 165 (2024), 107338.

[72] Deze Wang, Zhouyang Jia, Shanshan Li, Yue Yu, Yun Xiong, Wei Dong, and Xiangke Liao. 2022. Bridging pre-trained models and downstream tasks for source code understanding. In *Proceedings of the 44th international conference on software engineering*. 287–298.

[73] Dingbang Wang, Yu Zhao, Sidong Feng, Zhaoxu Zhang, William GJ Halfond, Chunyang Chen, Xiaoxia Sun, Jiangfan Shi, and Tingting Yu. 2024. Feedback-driven automated whole bug report reproduction for android apps. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1048–1060.

[74] Xinchen Wang, Pengfei Gao, Xiangxin Meng, Ruida Hu, Chao Peng, Yun Lin, and Cuiyun Gao. 2024. AEGIS: An Agent-based Framework for Bug Reproduction from Issue Descriptions. (2024).

[75] Xin Wang, Yasheng Wang, Yao Wan, Fei Mi, Yitong Li, Pingyi Zhou, Jin Liu, Hao Wu, Xin Jiang, and Qun Liu. 2022. Compilable Neural Code Generation with Compiler Feedback. In *Findings of the Association for Computational Linguistics: ACL 2022*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 9–19. doi:10.18653/v1/2022.findings-acl.2

[76] Mohammad Wardat, Wei Le, and Hridesh Rajan. 2021. Deeplocalize: Fault localization for deep neural networks. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 251–262.

[77] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[78] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1482–1494.

[79] Linghao Zhang, Hongyi Zhang, Chong Wang, and Peng Liang. 2024. RAG-Enhanced Commit Message Generation. arXiv:2406.05514 [cs.SE] https://arxiv.org/abs/2406.05514

[80] Xiaoyu Zhang, Juan Zhai, Shiqing Ma, and Chao Shen. 2021. Autotrainer: An automatic dnn training problem detection and repair system. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 359–371.

[81] Yuhao Zhang, Yifan Chen, Shing-Chi Cheung, Yingfei Xiong, and Lu Zhang. 2018. An empirical study on tensorflow program bugs. In *ISSTA*. 129–140.

[82] Yuhao Zhang, Luyao Ren, Liqian Chen, Yingfei Xiong, Shing-Chi Cheung, and Tao Xie. 2020. Detecting numerical bugs in neural network architectures. In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*. 826–837.

[83] Yu Zhao, Ting Su, Yang Liu, Wei Zheng, Xiaoxue Wu, Ramakanth Kavuluru, William GJ Halfond, and Tingting Yu. 2022. Recdroid+: Automated end-to-end crash reproduction from bug reports for android apps. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 3 (2022), 1–33.

[84] Yu Zhao, Tingting Yu, Ting Su, Yang Liu, Wei Zheng, Jingzhi Zhang, and William GJ Halfond. 2019. Recdroid: automatically reproducing android application crashes from bug reports. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 128–139.